JORHAT ENGINEERING COLLEGE

Jorhat, Assam

Department of Computer Science and Engineering



PROJECT REPORT

AEROHEALTH

Remaining Useful Life Prediction of Turbofan Engines

Comparing machine-learning and deep-learning models using the NASA C-MAPSS FD001 dataset

Submitted by

SOUMIK SARKAR

B.Tech. - Computer Science and Engineering

Jorhat Engineering College | Assam Science and Technology University

Project developed as a practical application of

NPTEL: Deep Learning - IIT Ropar/ IIT Madras (Jan-Apr 2026)

October 2026

Acknowledgement

I would like to thank the faculty members of the Computer Science and Engineering Department at Jorhat Engineering College for the support and learning environment that helped me complete this project.

I am thankful to the NPTEL team and IIT Ropar and IIT Madras for the Deep Learning course. What I learned in the course encouraged me to build a project and try the ideas on a real modelling task.

I also acknowledge NASA for providing the C-MAPSS engine dataset and the developers of the open-source Python libraries used in this work. I am grateful to my family and friends for their encouragement.

This report describes the work in my project notebook and the results saved from the experiments. Any limits of the current work are explained honestly in the report.

Soumik Sarkar

Abstract

Remaining Useful Life (RUL) prediction means estimating how many operating cycles a machine may have left before the end of its useful life. In this project, called AeroHealth, I use the NASA C-MAPSS FD001 turbofan-engine dataset to predict RUL from past sensor readings. I compare four traditional machine-learning models with three deep-learning models: LSTM, GRU and GRU with temporal attention.

The data are arranged into windows of 30 cycles. I split the training data by engine, remove features that hardly change, and scale the inputs using only the training engines. I test each neural model with five random seeds and compare the results using MAE, RMSE, R-squared and the NASA score.

On the official FD001 test set, GRU gives the lowest average MAE (10.78 cycles) and the lowest average NASA score (340.45). GRU with attention gives the lowest average RMSE (14.75 cycles), but the difference from GRU is very small. Ridge Regression using 30-cycle windows also performs well. Overall, the results show that a more complex model does not always give a clear improvement.

Keywords: Remaining Useful Life, predictive maintenance, C-MAPSS FD001, LSTM, GRU, attention, time-series prediction.

Table of Contents

Acknowledgement ii

Abstract iii

Table of Contents iv

List of Figures and Tables v

CHAPTER 1 Introduction 1
1.1 Background 1
1.2 Problem to Solve 1
1.3 Objectives 1
1.4 Project Scope 1
1.5 Report Structure 1

CHAPTER 2 Background and Key Ideas 2
2.1 Predictive Maintenance and RUL 2
2.2 Why Use Time Windows? 2
2.3 Recurrent Neural Networks 2
2.4 Temporal Attention 2
2.5 Questions for the Experiment 2

CHAPTER 3 Dataset and Data Preparation 3
3.1 Dataset Overview 3
3.2 RUL Target and 125-Cycle Cap 3
3.3 Engine Split, Feature Selection and Scaling 3
3.4 Creating 30-Cycle Windows 4

CHAPTER 4 Methods and Models 5
4.1 Project Steps 5
4.2 Traditional Machine-Learning Models 5
4.3 Deep Learning Models 5
4.4 Training Setup 6
4.5 Evaluation Metrics 6

CHAPTER 5 Implementation 7
5.1 Tools and Libraries 7
5.2 Implementation Steps 7
5.3 Saved Files and Repeatability 7

CHAPTER 6 Results and Discussion 8
6.1 Test Results 8
6.2 Main Findings 8
6.3 Visual Analysis 9
6.4 What the Results Mean 12

CHAPTER 7 Limitations and Future Improvements 13
7.1 Limitations 13
7.2 Future Work 13

CHAPTER 8 Conclusion 14
References 15
Appendix A: Sample Test Predictions 16
Appendix B: NPTEL Course Certificate 17

List of Figures and Tables

Reference	Title	Page
Table 3.1	Summary of the FD001 dataset	3
Table 4.1	Traditional model inputs and settings	5
Table 4.2	Deep-learning model designs	5
Table 4.3	Evaluation scores and what they mean	6
Table 6.1	FD001 test results (average and standard deviation where available)	8
Figure 6.1	Model comparison: MAE, RMSE and R-squared	9
Figure 6.2	Training and validation loss (seed 0)	9
Figure 6.3	Predicted and actual test RUL (seed 0)	10
Figure 6.4	Actual and predicted RUL for validation engine 7	10
Figure 6.5	Test prediction errors (seed 0)	11
Figure 6.6	Attention weights and heatmap	11
Figure 6.7	RMSE changes across five runs	12
Table A.1	First ten examples from the test predictions	16

CHAPTER 1
INTRODUCTION

1.1 Background

Predictive maintenance uses data from sensors to check the condition of equipment and help plan maintenance before a failure happens. Remaining Useful Life (RUL) is an estimate of how many operating cycles are left. This project uses engine sensor readings to predict RUL and compares different models to see which ones work best on the FD001 test data.

1.2 Problem to Solve

The task is to use recent sensor readings and operating settings to estimate the engine's capped RUL at the end of that period. This is a regression problem because the model predicts a number rather than a category.

1.3 Objectives

- Prepare the FD001 data and create input windows of 30 cycles.
- Compare four traditional models with LSTM, GRU and GRU with attention.
- Measure results using MAE, RMSE, R-squared and the NASA score.
- Train each neural model with five random seeds and study the results and plots.

1.4 Project Scope

This project is a comparison of existing models, not a new learning algorithm. It brings together data preparation, window building, traditional models, recurrent neural networks, attention and several ways to measure prediction quality.

1.5 Report Structure

The report first explains the main ideas, then describes the dataset, data preparation, models and implementation. The later chapters discuss the results, limits, possible improvements and conclusion.

CHAPTER 2
BACKGROUND AND KEY IDEAS

2.1 Predictive Maintenance and RUL

Predictive maintenance uses sensor readings to understand the condition of a machine. RUL prediction gives a number for the estimated life left. For example, a prediction of 40 cycles means the model estimates about 40 operating cycles remain under the target rule used in this dataset. The result depends on the data and on how RUL is defined.

NASA's C-MAPSS dataset contains simulated engine sensor records. The training engines have records up to failure, while the test engines have shorter records and separate RUL labels [1]. FD001 covers one operating condition and one simulated fault type, so it is a useful starting point for sequence prediction.

2.2 Why Use Time Windows?

One time step gives sensor readings from a single cycle. A window joins several nearby cycles so the model can learn how the readings change over time. In AeroHealth, each input window contains 30 cycles, and the target is the RUL at the end of the window. Windows are built separately for each engine, so records from different engines are never joined together.

2.3 Recurrent Neural Networks

LSTM networks use a memory mechanism to learn patterns across a sequence [2]. GRU networks also use gates to control what information is kept from earlier time steps [3]. Both can work with ordered sensor readings, where the order of the data matters.

2.4 Temporal Attention

The GRU + Attention model produces a representation for each time step and learns a weight for each one. It combines these weighted representations to make the final RUL prediction. The aim is to let the model put more weight on time steps that are useful for the task. The attention plot helps us inspect the model, but it does not prove that a time step causes engine damage.

2.5 Questions for the Experiment

The experiment looks at three questions: Do the neural models improve on traditional models? Does giving a traditional model the full 30-cycle window help? Does attention give a steady improvement over a normal GRU when the model is trained with different seeds?

CHAPTER 3
DATASET AND DATA PREPARATION

3.1 Dataset Overview

The notebook reads three FD001 files: train_FD001.txt, test_FD001.txt and RUL_FD001.txt. Each row contains an engine ID, a cycle number, three operating settings and 21 sensor readings. The standard FD001 training set contains records for 100 engines, and the test set contains another 100 engines [1]. The test records stop before failure, and the separate RUL file gives the remaining-life labels for those test engines.

Item	Details
Dataset subset	NASA C-MAPSS FD001
Training trajectories	100 engines with records up to failure
Test trajectories	100 engines with partial records and separate RUL labels
Variables per row	Engine ID, cycle, 3 operating settings and 21 sensor readings
Prediction target	Remaining Useful Life, measured in cycles
Input sequence length	30 cycles in a row
Target cap	125 cycles

The notebook can download the files from the listed mirrors if they are not already available. This report was prepared from the notebook and its saved output files; I did not reload the original text files separately while preparing the report.

3.2 RUL Target and 125-Cycle Cap

For each training engine, RUL at a given cycle is found by subtracting that cycle from the engine's last training cycle. The notebook limits values to a maximum of 125 cycles:

$$\mathbf{RUL(i,t) = min(T(i) - t, 125)}$$

Here, T(i) means the last cycle recorded for engine i. For the official test set, the RUL values come from RUL_FD001.txt and are capped at 125 as well. Predictions are also limited to the range from 0 to 125 cycles before the scores are calculated.

3.3 Engine Split, Feature Selection and Scaling

The 100 training engines are shuffled using seed 42. Eighty engines are used for training and 20 for validation. The split is done by engine, not by individual rows, so windows from the same engine are less likely to appear in both groups.

Features that barely change are removed using the training engines only. The scaler is also fitted using only the training data, then applied to the validation and test data. This helps prevent the validation and test data from affecting preprocessing.

3.4 Creating 30-Cycle Windows

For each engine, the rows are sorted by cycle. Once 30 cycles are available, the notebook creates a window from those recent cycles and uses the RUL at the last cycle as the target. Training and validation can have many windows per engine. For the official test set, only the last 30 cycles of each test engine are used, giving one prediction for each of the 100 test engines.

For neural training, the RUL target is divided by 125 so that it falls between 0 and 1. The predictions are changed back to cycles before evaluation. This target scaling is separate from the scaling applied to the input features.

CHAPTER 4
METHODS AND MODELS

4.1 Project Steps

1. Load the FD001 training, test and test-RUL files.
2. Create capped RUL labels and split the training engines into training and validation groups.
3. Keep useful features and scale them using training data only.
4. Create input windows of 30 cycles.
5. Train the traditional and deep-learning models.
6. Test the models, save the scores and export prediction tables and plots.

4.2 Traditional Machine-Learning Models

The project tests four traditional models. Linear Regression and Random Forest use the readings from the latest cycle only. Ridge Regression and the second Random Forest model use the full 30-cycle window, converted into one long list of values. This comparison checks whether the recent history helps traditional models make better predictions.

Model	Input data	Setup and purpose
Linear Regression	Latest cycle readings	Simple linear model using current readings
Ridge Regression (window)	All 30 cycles joined into one long list	Ridge with alpha = 10; uses recent history
Random Forest	Latest cycle readings	200 trees; learns non-linear patterns from current readings
Random Forest (window)	All 30 cycles joined into one long list	200 trees; uses recent history and selects a square-root number of features

4.3 Deep Learning Models

Model	Model design in the notebook	Purpose
LSTM	LSTM(64, sequences) -> dropout 0.2 -> LSTM(32) -> dropout 0.2 -> Dense(32, ReLU) -> Dense(1)	Learn patterns across time using a memory layer [2].
GRU	GRU(64, sequences) -> dropout 0.2 -> GRU(32) -> dropout 0.2 -> Dense(32, ReLU) -> Dense(1)	Try another recurrent model with a different way to update its memory [3].
GRU + Attention	GRU(64, sequences) -> dropout 0.2 -> GRU(32, sequences) -> dropout 0.2 -> temporal attention -> Dense(32, ReLU) -> Dense(1)	Give different time steps different weights before predicting RUL.

4.4 Training Setup

Setting	Value used
Window length	30 cycles
Maximum RUL value	125 cycles
Random seeds for neural models	42, 43, 44, 45 and 46 (five runs per architecture)
Maximum number of epochs	60
Batch size	128
Optimiser	Adam, learning rate 0.001
Training loss	Mean Squared Error on RUL divided by 125
Dropout	Dropout rate 0.2
Early stopping	Watch validation loss; stop after 12 epochs without improvement and keep the best weights
Learning-rate change	Cut learning rate by half after 5 epochs without improvement; minimum 0.000001

Each traditional model is trained once with the fixed settings. Each neural model is trained five times using seeds 42, 43, 44, 45 and 46. The result table reports the average score and standard deviation across the five runs.

4.5 Evaluation Metrics

Metric	Meaning	Better result
MAE	Average absolute difference between predicted and actual RUL, in cycles.	Lower
RMSE	Error measure that gives more weight to large mistakes.	Lower
R-squared (R^2)	Shows how well the predictions explain changes in the actual RUL.	Higher
NASA score	A special score that penalises overestimating RUL more strongly in this notebook.	Lower

MAE shows the average absolute error. RMSE gives more weight to large mistakes, while R-squared shows how well predictions match the actual values. The NASA score gives a larger penalty when RUL is overestimated.

CHAPTER 5
IMPLEMENTATION

5.1 Tools and Libraries

Tool / library	How it is used
Python and Jupyter Notebook	Runs the project steps, models, evaluation and plots.
NumPy and Pandas	Handles numbers and tables, loads data and prepares result summaries.
scikit-learn	Scales features, trains traditional models and calculates scores.
TensorFlow / Keras	Builds and trains LSTM, GRU and the attention model.
Matplotlib	Creates the training curves, prediction plots, error plots and summary charts.

5.2 Implementation Steps

The notebook is divided into steps: load the data, prepare it, define the scores, train the traditional models, train the neural models, collect results, make plots and save the outputs. Predictions are limited to the chosen RUL range before the scores are calculated. The code saves summary tables and one prediction row for each test engine.

The notebook saves predictions and training history from seed 0 for the diagnostic plots. The main comparison chart uses average scores and standard deviations across five neural runs. The prediction scatter plots, one-engine trajectory, error plots and attention plots use a single run. This is why some plots may not show exactly the same values as the main result table.

5.3 Saved Files and Repeatability

- results_per_seed.csv: scores for each individual run.
- results_table.csv and results_table_numeric.csv: summary tables of the results.
- test_predictions.csv: actual capped RUL and model predictions for all 100 test engines.
- fig1_model_comparison.png to fig7_seed_variability.png: the seven result plots.

Using fixed seeds makes runs easier to compare, but results can still change slightly with different software versions or computer hardware. This report describes the settings and outputs in the supplied notebook; it does not claim that every computer will produce exactly the same numbers.

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Test Results

Table 6.1 shows the model results on the official FD001 test set. The traditional models were each run once. For LSTM, GRU and GRU + Attention, the table shows the average and standard deviation from five runs.

Model	Test MAE ↓	Test RMSE ↓	Test R^2 ↑	Test NASA ↓
Linear Regression	16.20	20.29	0.744	1000.40
Ridge (window)	11.85	14.76	0.864	361.38
Random Forest	12.69	17.62	0.807	885.87
Random Forest (window)	12.59	17.20	0.816	747.96
LSTM	11.25 ± 0.29	15.11 ± 0.15	0.858 ± 0.003	366.06 ± 36.66
GRU	10.78 ± 0.57	14.85 ± 0.60	0.862 ± 0.011	340.45 ± 35.20
GRU + Attention	11.01 ± 0.46	14.75 ± 0.60	0.864 ± 0.011	408.24 ± 72.05

Table 6.1. Test results on FD001. Lower values are better for MAE, RMSE and NASA score. A higher R-squared is better.

6.2 Main Findings

6.2.1 GRU has the lowest MAE

GRU has an average MAE of 10.78 cycles, the lowest among the models tested. This means its predictions have the smallest average absolute error across the test engines. It does not mean that every prediction is within 10.78 cycles of the actual value.

6.2.2 The RMSE scores are almost the same

GRU + Attention has an average RMSE of 14.75 cycles. This is only 0.10 cycles lower than GRU at 14.85, and almost the same as Ridge Regression with windows at 14.76. This difference is very small compared with the changes between different neural runs, so the results do not show a clear advantage from adding attention.

6.2.3 Ridge Regression with windows is a strong baseline

Ridge Regression using the full 30-cycle window has an RMSE of 14.76 and an R-squared of 0.864. Its RMSE is almost the same as GRU + Attention. This shows that a simple model can work well when it is given a useful representation of recent history.

6.2.4 No model is best for every score

GRU has the best MAE and NASA score. GRU + Attention has the lowest average RMSE and the highest rounded R-squared. Since the winner changes depending on the score, it is better to explain the strengths of each model instead of saying one model wins in every way.

6.3 Visual Analysis

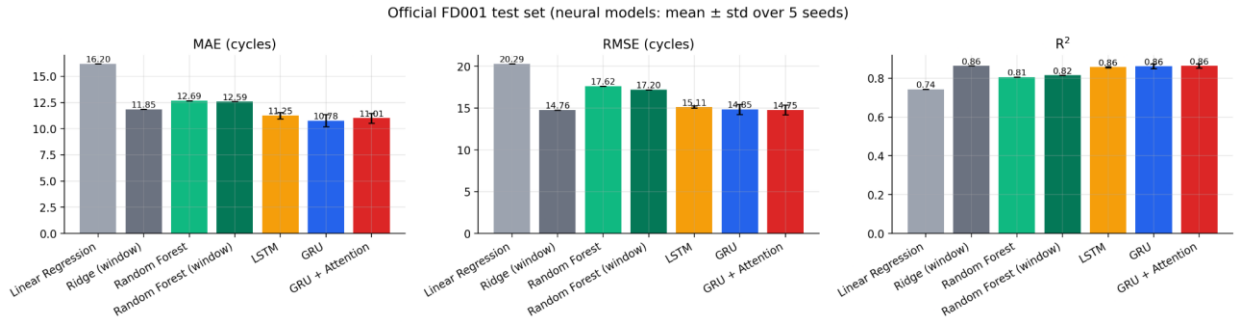


Figure 6.1. Comparison of test-set MAE, RMSE and R-squared across seven models.

The chart shows the main pattern from the table: GRU has the lowest MAE, while Ridge with windows and GRU + Attention have almost the same RMSE. The error bars also show that neural-model results change across the five runs.

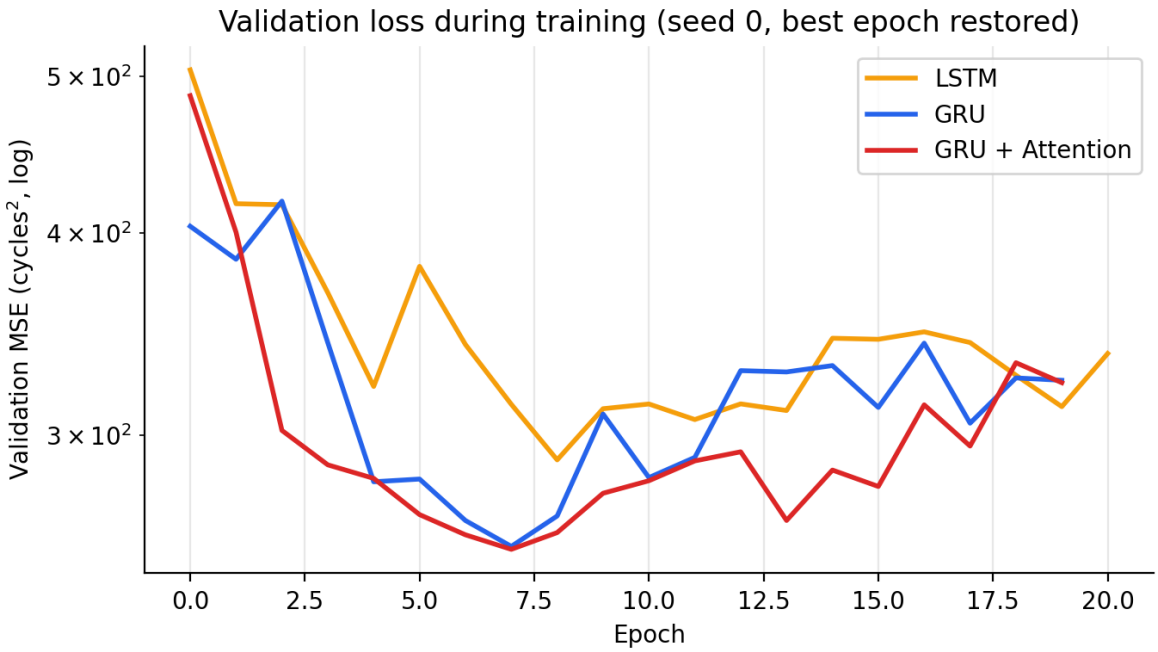


Figure 6.2. Validation loss during training for seed 0; the best model weights were kept.

Validation loss drops early in training but changes up and down later. Early stopping keeps the model weights from the epoch with the best validation loss. This figure shows one run, not the average of five runs.

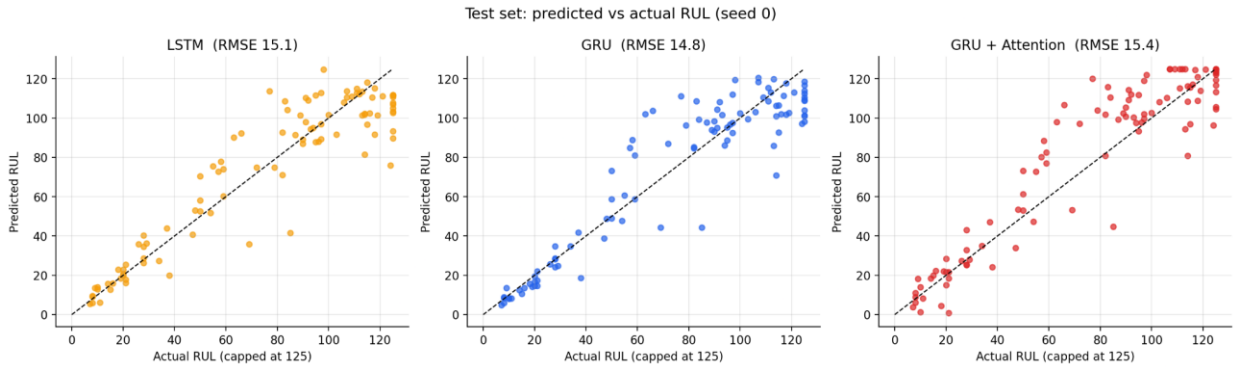


Figure 6.3. Predicted versus actual capped RUL for the 100 FD001 test engines (seed 0).

In this plot, points closer to the diagonal line show more accurate predictions. The spread of points shows that some engines are easier to predict than others. Values at actual RUL = 125 are capped, so they do not show the exact uncapped RUL.

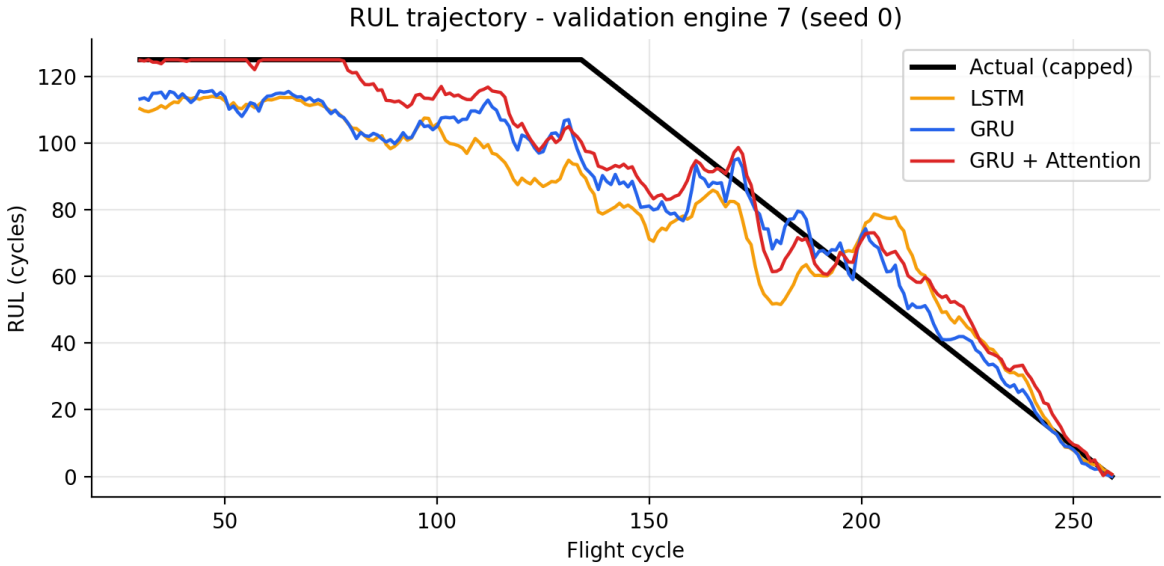


Figure 6.4. Actual and predicted RUL for validation engine 7 over its cycles.

This plot follows one validation engine over time. It shows when the predictions follow the general RUL pattern and when they move away from it. One engine is only an example; the test scores across all 100 engines give a broader view.

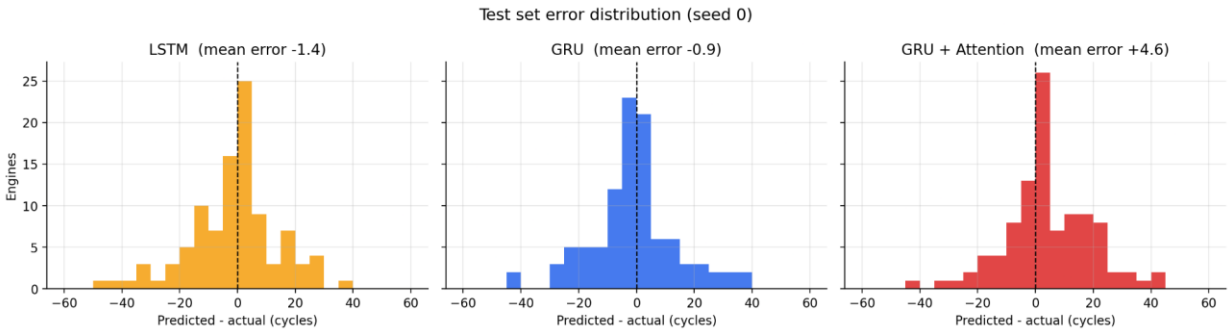


Figure 6.5. Test prediction errors (predicted RUL minus actual RUL) for seed 0.

The error is predicted RUL minus actual RUL. A positive value means the model predicted too much remaining life; a negative value means it predicted too little. In this run, GRU has a mean error closer to zero, while GRU + Attention tends to overestimate more.

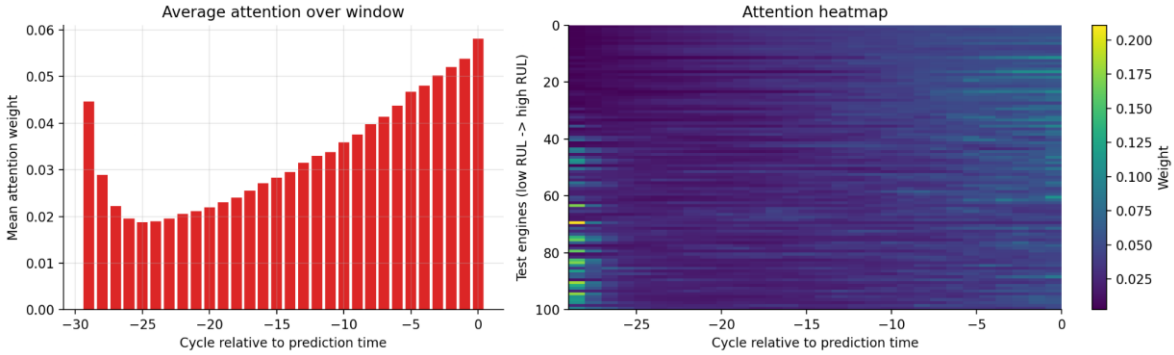


Figure 6.6. Average attention weights and the attention heatmap for GRU + Attention.

The attention plot shows how the model shares its weight across the 30 time steps. In this run, the model gives more average weight to the more recent steps. This describes what the trained model learned; it does not prove which readings are physically responsible for engine wear.

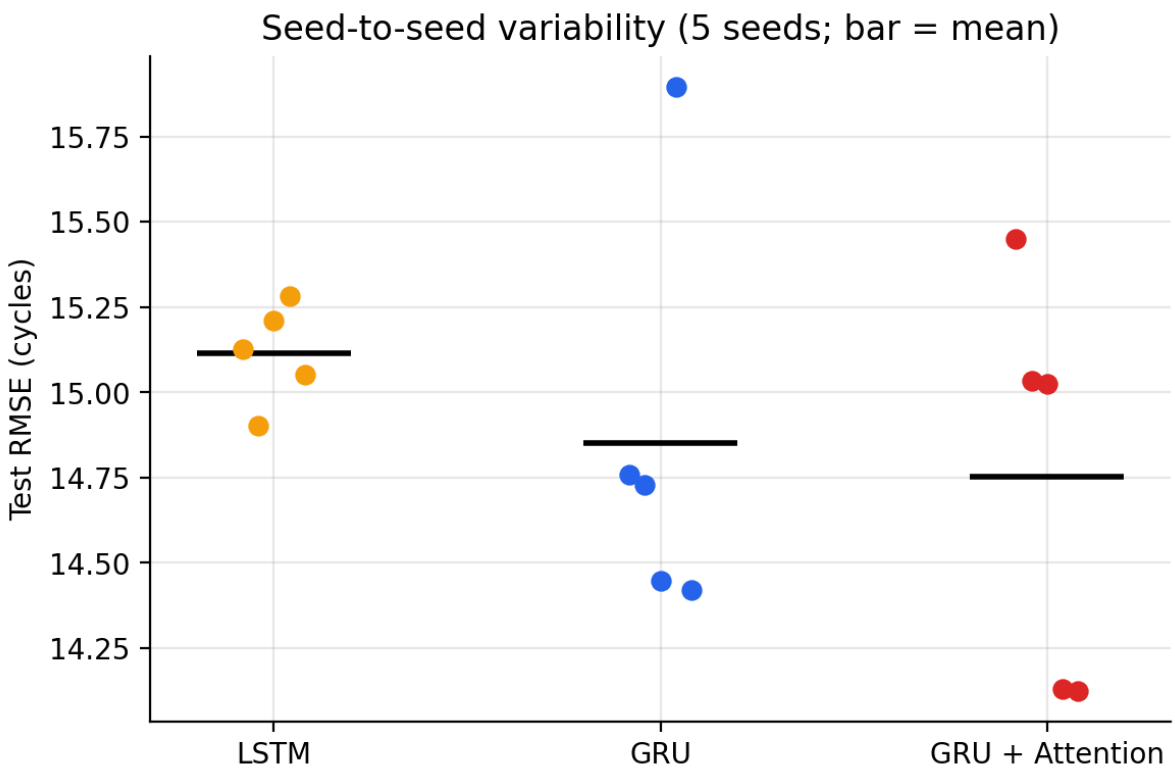


Figure 6.7. Test RMSE across five runs for LSTM, GRU and GRU + Attention.

The plot shows that the RMSE changes from one training run to another. GRU and GRU + Attention both have a standard deviation of about 0.60 cycles. This is much larger than their 0.10-cycle difference in average RMSE, so that small gap should be treated with care.

6.4 What the Results Mean

The results show that GRU performs well on MAE and the NASA score, but a more complex model does not always perform better on every measure. GRU + Attention has the lowest RMSE by a very small amount. Its result is almost tied with Ridge Regression using windows, so the current experiment does not show that attention gives a clear improvement.

The different plots help explain the scores. The summary table compares overall performance, the error plots show overestimates and underestimates, the trajectory plot follows one engine, and the repeated runs show how much results can change. Together, these give a fuller picture than one score alone.

CHAPTER 7
LIMITATIONS AND FUTURE IMPROVEMENTS

7.1 Limitations

- FD001 uses simulated engine data with one operating condition and one fault type. Good results on this subset do not prove that the model will work just as well on real engines or other C-MAPSS subsets.
- Each neural model was trained with five seeds. This gives an initial idea of result variation, but more runs would be needed for stronger conclusions.
- The difference in average RMSE between GRU and GRU + Attention is very small compared with the variation between runs. These results do not prove that attention reliably improves GRU.
- The prediction scatter, trajectory, error and attention plots use seed 0, while the main result table averages five runs. The two types of results should be described separately.
- RUL is limited to 125 cycles. This makes the target easier to compare, but early-life predictions show the capped value rather than the exact remaining life.
- This project has not been tested as a real maintenance system. It does not yet cover prediction uncertainty, changes in sensor quality or the cost of different maintenance decisions.

7.2 Future Work

- Run the notebook from start to finish and check that all tables and plots match the saved results.
- Compare GRU and GRU + Attention using the results from each matching seed before claiming that one is better.
- Try different window lengths, network sizes, dropout values and learning rates using validation data only. Keep the official test set for the final check.
- Test other target scales and loss functions, while keeping model selection separate from final test evaluation.
- Test FD002, FD003 and FD004 to see how the models work with more operating conditions and fault types.
- Add ways to show how sure the model is about a prediction and consider measures related to maintenance decisions.
- Compare the results with published FD001 results using the same target and evaluation rules before making claims about being better than other methods.

CHAPTER 8
CONCLUSION

AeroHealth uses ideas from the NPTEL Deep Learning course to solve a time-series prediction problem. The project uses NASA C-MAPSS FD001, makes 30-cycle sensor windows, prepares the data using training engines only, compares traditional models with LSTM, GRU and GRU with attention, and checks the results with four scores.

GRU has the lowest average MAE (10.78 cycles) and the lowest average NASA score (340.45) among the models tested. GRU + Attention has the lowest average RMSE (14.75 cycles), but its score is almost the same as GRU and Ridge Regression with windows. Ridge Regression is therefore an important comparison model, even though it is not a deep-learning model.

The main lesson is that a more complicated model does not always give a clear improvement. The best model depends on the score we care about and how steady the results are across runs. More tests are needed before making broader claims, including tuning with validation data and testing other C-MAPSS subsets.

What I Learned

Building this project helped me connect the ideas from the Deep Learning course with a full experiment. I learned how to prepare time-series data, train and compare models, read different evaluation scores, and explain results using plots. I also learned that good data preparation and fair comparisons matter just as much as the model design.

References

[1] NASA Open Data Portal, “CMAPSS Jet Engine Simulated Data.” Dataset description for C-MAPSS trajectories and the FD001 subset. Available at: https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data (accessed October 2026).

[2] TensorFlow, Keras API documentation, used as the implementation framework for recurrent layers, optimisation and training callbacks. Available at: https://www.tensorflow.org/guide/keras.

[3] Scikit-learn, user guide and API documentation for StandardScaler, LinearRegression, Ridge, RandomForestRegressor and regression metrics. Available at: https://scikit-learn.org/stable/.

[4] NPTEL, “Deep Learning - IIT Ropar/IIT Madras,” online course, January-April 2026. Course completion certificate provided by the author.

[5] OpenAI, “ChatGPT,” AI language model used as a supporting tool for information gathering, clarification of deep learning concepts, and assistance with project documentation.

Appendix A: Sample Test Predictions

This table shows the first ten rows from the saved test_predictions.csv file. RUL is capped at 125 cycles. The LSTM, GRU and attention predictions shown here are from seed 0, so they are examples rather than averages across the five runs.

Engine	Actual capped RUL	LSTM prediction	GRU prediction	GRU + Attention prediction
1	112	112.2	113.0	125.0
2	98	124.8	119.4	121.9
3	69	35.8	44.4	53.2
4	82	92.8	84.6	101.8
5	91	110.3	104.4	114.3
6	93	94.4	101.6	100.4
7	91	98.0	95.0	109.2
8	95	111.6	95.3	111.9
9	111	112.9	108.5	115.2
10	96	87.9	96.6	97.8

Table A.1. First ten examples from the saved test predictions.

The file test_predictions.csv contains predictions for all 100 test engines. It can be used to check individual cases alongside the overall model scores.

Appendix B: NPTEL Course Certificate

The certificate below shows that I completed the NPTEL online course "Deep Learning - IIT Ropar/IIT Madras" in January-April 2026. It is included to show the course that inspired this project; it does not mean that NPTEL has approved or tested the AeroHealth model.

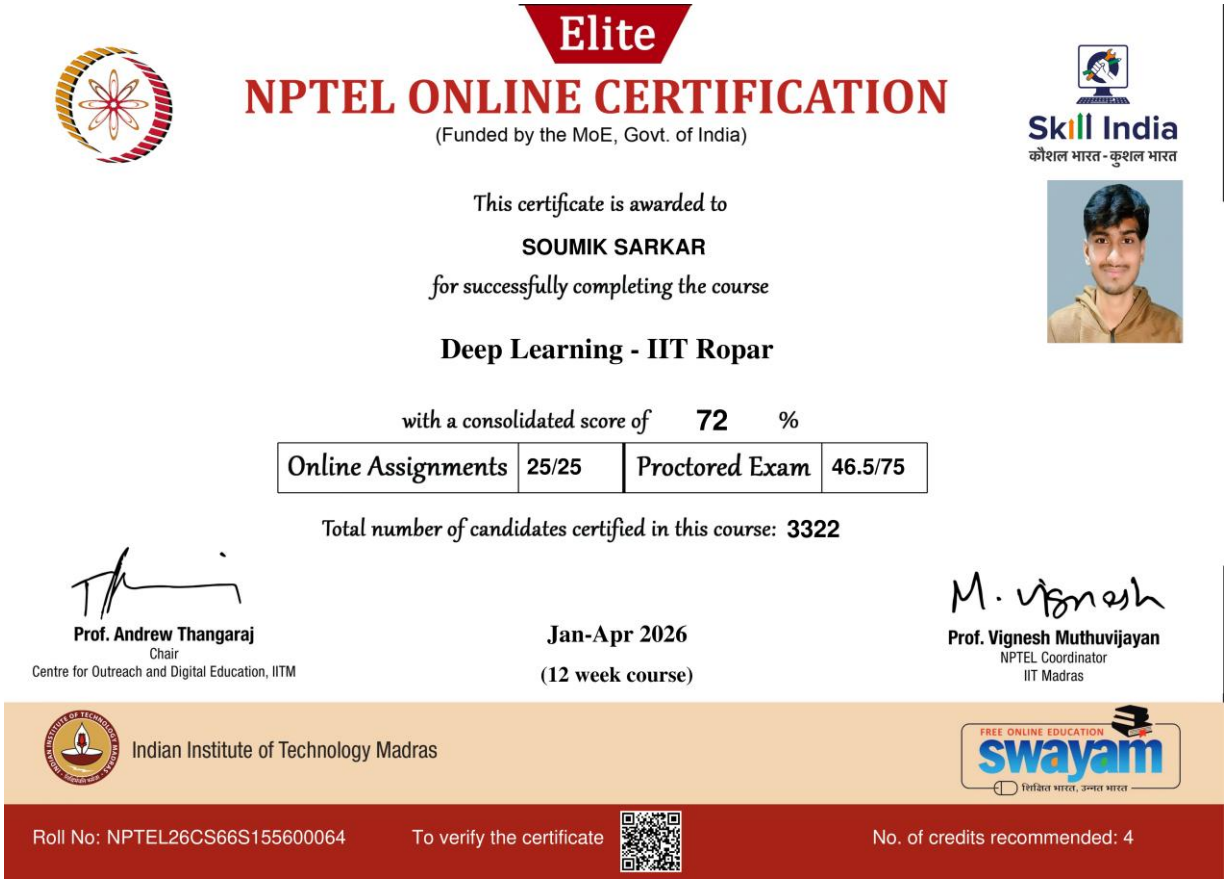


Figure B.1. NPTEL Deep Learning - IIT Ropar/IIT Madras course certificate.